

When we say, “Python runs a program” or “Python will raise an exception,” we’re talking about the *Python interpreter*—the actual software that reads the text of a *.py* file and carries out its instructions. When we say, “the Python interpreter,” we’re almost always talking about *CPython*, the Python interpreter maintained by the Python Software Foundation, available at <https://www.python.org>. CPython is an *implementation* of the Python language—that is, software created to follow a specification—but there are others. Although CPython is written in the C programming language, *Jython* is written in Java for running Python scripts that are interoperable with Java programs. *PyPy*, a *just-in-time compiler* for Python that compiles as programs execute, is written in Python.

All of these implementations run source code written in the *Python programming language*, which is what we mean when we say, “This is a Python program” or “I’m learning Python.” Ideally, any Python interpreter can run any source code written in the Python language; however, in the real world there’ll be some slight incompatibilities and differences between interpreters. CPython is called the Python language’s *reference implementation* because if there’s a difference between how CPython and another interpreter interpret Python code, CPython’s behavior is considered canonical and correct.

## ***Garbage Collection***

In many early programming languages, a programmer had to instruct the program to allocate and then deallocate, or free, memory for data structures as needed. Manual memory allocation was the source of numerous bugs, such as *memory leaks* (where programmers forgot to free memory) or *double-free bugs* (where programmers freed the same memory twice, leading to data corruption).

To avoid these bugs, Python has *garbage collection*, a form of automatic memory management that tracks when to allocate and free memory so the programmer doesn’t have to. You can think of garbage collection as memory recycling, because it makes memory available for new data. For example, enter the following into the interactive shell:

---

```
>>> def someFunction():  
...     print('someFunction() called.')
```